



Tema 5 - Microarquitectura - Pipeline Superescalar En RISC

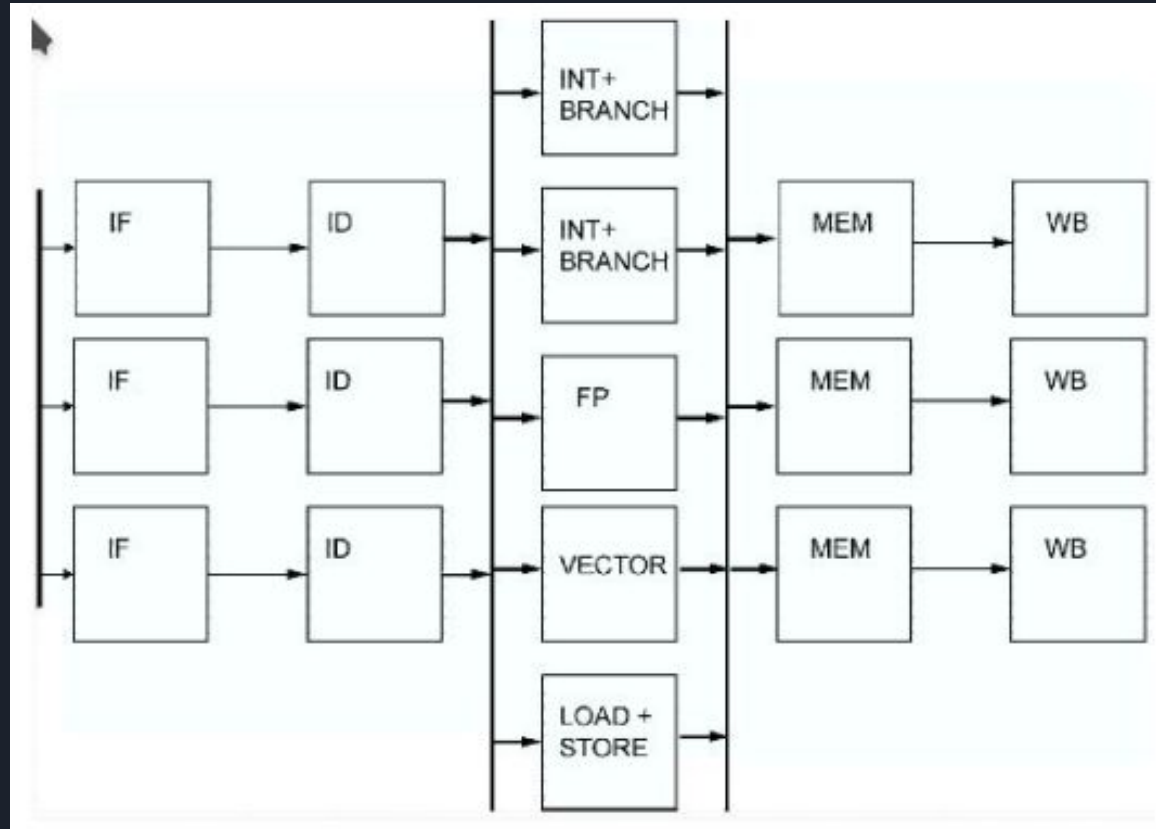
Cátedra: Organización del Computador II
Lic. Juan Pablo Moreno



Pipeline

- Paralelismo a nivel de instrucción
- Los riesgos (estructurales, de datos y control) producen stalls
- $CPI = CPI_{ideal} + stalls_{estructurales, de\ datos, de\ control}$
- Con las técnicas vistas, a lo sumo se emite una instrucción por ciclo
 - Forwarding
 - Predicción simple de saltos
 - Planificación Estática
- Superar esa barrera requiere enviar más de una instrucción por ciclo y por ende múltiples pipelines paralelos

Superescalar



Dependencias de datos

- Dependencia de datos "verdadera"
 - El resultado de la instrucción i es usado por la instrucción j (j depende de i)
 - La instrucción j depende de k que depende de i (transitiva).
- Las instrucciones dependientes no se pueden ejecutar simultáneamente o solaparse por completo
- Hay una cadena de riesgos de datos
- El efecto de la dependencia se debe preservar

```
Loop:    L.D    F0,0(R1)    ;F0=array element
          ADD.D  F4,F0,F2    ;add scalar in F2
          S.D    F4,0(R1)    ;store result
          DADDUI R1,R1,#-8    ;decrement pointer 8 bytes
          BNE    R1,R2,LOOP  ;branch R1!=R2
```

Dependencias de nombres

- Dos instrucciones usan el mismo registro o posición de memoria (nombre)
- No hay flujo de datos entre las instrucciones
- Antidependencia
 - La instrucción i lee un registro que es escrito a continuación por la instrucción j
 - El ordenamiento debe preservarse para que i no lea un valor distinto del que debería
- Dependencia de salida
 - La instrucción i y la instrucción j escriben en la misma ubicación.
 - El valor debe ser el de la última escritura en la secuencia del programa
- Estas dependencias “no verdaderas” se pueden evitar renombrando los registros

Predicción de saltos

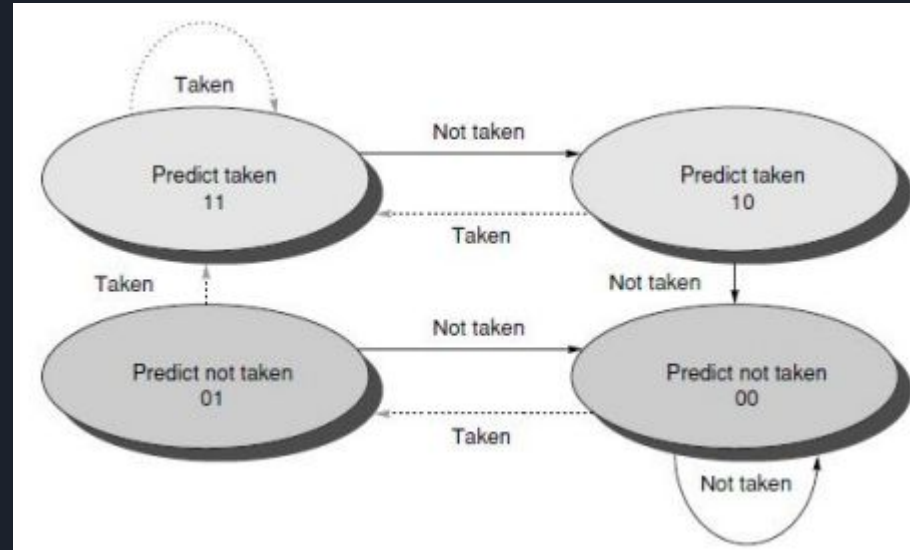
- Estática
 - Saltos demorados
 - Predicted taken
 - Predicted not taken
- Dinámica
 - Branch Prediction Buffer
 - Usa una tabla con parte de la dirección de la instrucción de salto y un bit que indica si se tomó o no
 - Cada salto actualiza el bit en la tabla
 - Impreciso
 - Se puede mejorar usando 2 bits para predecir el salto

Dirección	Bit
0x34123	1
0x19923	0
0x21339	1

Predicción dinámica de saltos usando dos bits

Tiene que haber dos saltos coincidentes para que cambie la predicción.

Dirección	Bits
0x34123	11
0x19923	10
0x21339	00





Planificación Dinámica

- Usando planificación estática la detección de riesgos produce stalls que frenan la ejecución hasta que la instrucción que lo produjo sale del pipeline
- Con planificación dinámica, el hardware reordena la ejecución para reducir stalls
- Detecta dependencias desconocidas en tiempo de compilación
- Tolera demoras impredecibles (× ejemplo fallos de caché)
- Las instrucciones ingresan al pipeline en orden, la ejecución debe comenzar apenas estén disponibles los operandos.
 - Ejecución fuera de orden
- Complica el manejo de excepciones

Categorías de Dependencias de datos

- RAW (Read After Write) La instrucción j lee el registro escrito por i
 - Dependencia verdadera
 - Se debe preservar el orden del programa
- WAW (Write After Write) Las dos instrucciones escriben el mismo registro
 - Dependencia de salida
 - Más de una etapa de escritura
 - Debe ejecutarse última la que iba según su orden
 - No se manifiesta en pipelines con planificación estática
- ● WAR (Write After Read) Lectura seguida de escritura
 - Antidependencia
 - La lectura no debe ver el resultado de la escritura posterior
 - No se manifiesta en pipelines con planificación estática
 - Puede aparecer al reordenar instrucciones

Ejecución Fuera de Orden

- Introduce riesgos WAR y WAW que no eran posibles con planificación estática.
- Las instrucciones pueden completarse fuera de orden
- Puede provocar excepciones imprecisas (el estado puede no corresponder a la ejecución en secuencia de las instrucciones)
 - Se completaron instrucciones posteriores a la instrucción que provoca la excepción
 - Puede haber instrucciones anteriores que no se completaron
- Fetch lee de una cola de instrucciones pendientes
- La etapa Decodificar se puede dividir en
 - Issue (Decodificar y chequear riesgos estructurales)
 - Leer operandos (Esperar a que no haya riesgos de datos y leer operandos)

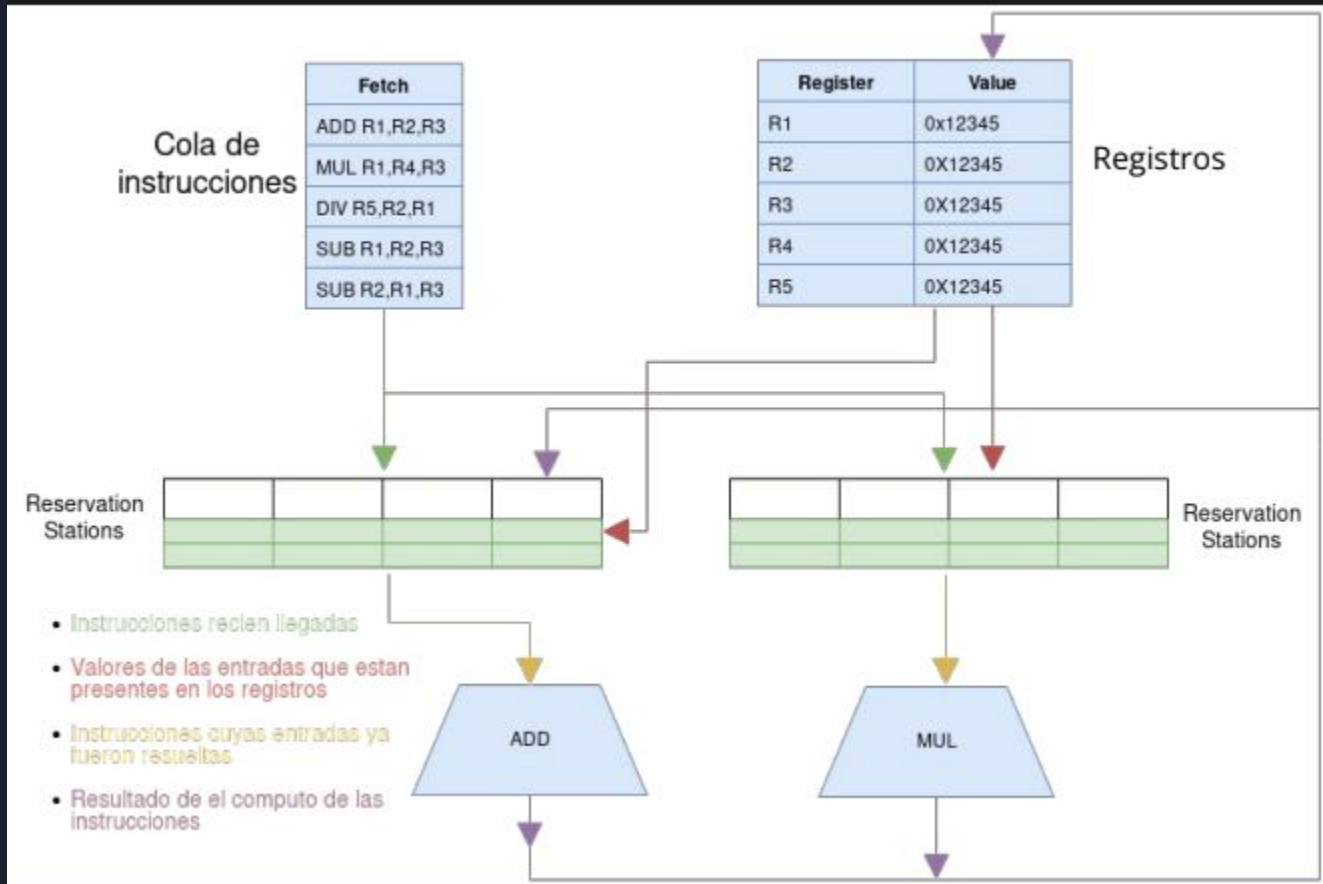


Método de Tomasulo

- Usado por primera vez en la IBM 360
- Determina qué Instrucciones tienen sus entradas disponibles
- Renombrado de registros
- Inicialmente se pensó para las unidades de punto flotante
- Muy similar a lo que se usa hoy en día a pesar de tener más de 40 años,
 - Ahora se aplica a todo no solo a FPU

Método de Tomasulo

- Detecta cuándo están disponibles los operandos de las instrucciones
 - Evita riesgos RAW
- Usa renombramiento de registros para eliminar WAW y WAR
- Reservation Stations (Registros intermedios) asociados a la instrucción
 - Renombra registros
 - Almacenan los valores apenas están disponibles
 - La instrucción se ejecuta cuando todos sus operandos están disponibles
 - El resultado se transmite a todas las Reservation Stations que lo están esperando
- Múltiples unidades funcionales para ejecutar instrucciones



Cola de instrucciones

Fetch
$R1 = R2 + R3$
$R4 = R1 - R2$
$R1 = R2 / R3$
$R2 = R4 + R1$

Registros

Register	Value
R1	3.14
R2	-1
R3	2.72
R4	0.71

RAT

Register	Value
R5	
R1	
R2	
R3	
R4	

Reservation Stations

ADD

MUL

Cola de instrucciones

Fetch
ADD R1,R2,R3
SUB R4,R1,R2
DIV R1,R2,R3

Registros

Register	Value
R1	3.14
R2	-1
R3	2.72
R4	0.71

RAT

Register	Value
R1	
R2	RS1
R3	
R4	

Reservation Stations

RS1	ADD	0.71 3.14
RS2		
RS3		

RS4		
RS5		

ADD

MUL

Cola de instrucciones

Fetch
ADD R1,R2,R3
SUB R4,R1,R2

Registros

Register	Value
R1	3.14
R2	-1
R3	2.72
R4	0.71

RAT

Register	Alias
R1	RS4
R2	RS1
R3	
R4	

Reservation Stations

RS1			
RS2	ADD	0.71	3.14
RS3			

RS4			
RS5	DIV	RS1	2.72

ADD

MUL

Cola de instrucciones

Fetch
ADD R1,R2,R3

Registros

Register	Value
R1	3.14
R2	-1
R3	2.72
R4	0.71

RAT

Register	Alias
R1	RS4
R2	RS1
R3	
R4	RS2

Reservation Stations

RS1	ADD	0.71 3.14
RS2	SUB	RS4 RS1
RS3		

RS4	DIV	RS1 2.72
RS5		

ADD

MUL



Ventajas

- No necesita un ordenamiento especial de las instrucciones para alcanzar alto rendimiento
- La ejecución fuera de orden permite continuar la ejecución ante fallos de caché
- Permite envío agresivo de instrucciones, simplifica la planificación
- Importante para la especulación por hardware



Ejecución especulativa

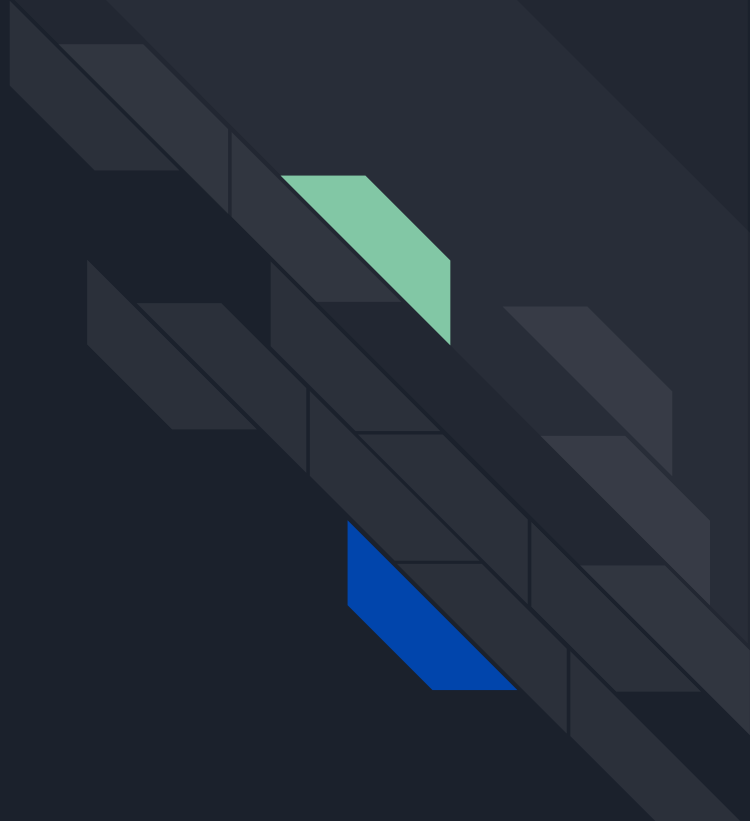
- Ejecutar fuera de orden, confirmando los resultados en el orden establecido por el programa
- Predecir saltos
- Ejecutar especulativamente hasta confirmar el salto
- Confirmar los resultados de las instrucciones completas (commit)
- Separar la ejecución de la confirmación del resultado requiere buffers
 - Reorder buffer (ROB)
- Las instrucciones que no se confirman se deben deshacer



Mejorando el IPC

- Dependencias de control $\Rightarrow$ Branch Prediction
- Dependencias de datos WAR & WAW $\Rightarrow$ Renombrado de Registros $\rightarrow$ Tomasulo
- Dependencias de datos RAW $\Rightarrow$ Ejecución fuera de Orden $\rightarrow$ Tomasulo
- Dependencias Estructurales $\Rightarrow$ Invertir en un procesador que soporte mayor caudal de instrucciones del tipo que nos afecta

Gracias!

An abstract graphic on the right side of the slide, consisting of several dark gray rectangular blocks arranged in a staggered, 3D-like pattern. Two blocks are highlighted: a light green one in the upper-middle section and a blue one in the lower-middle section.